SMART WEATHER MONITORING STATION

A Menu-Driven Environmental Monitoring System Using Arduino, DHT11, LDR, LCD, LEDs and Buzzer

Submitted To: *Embeduino Labs*

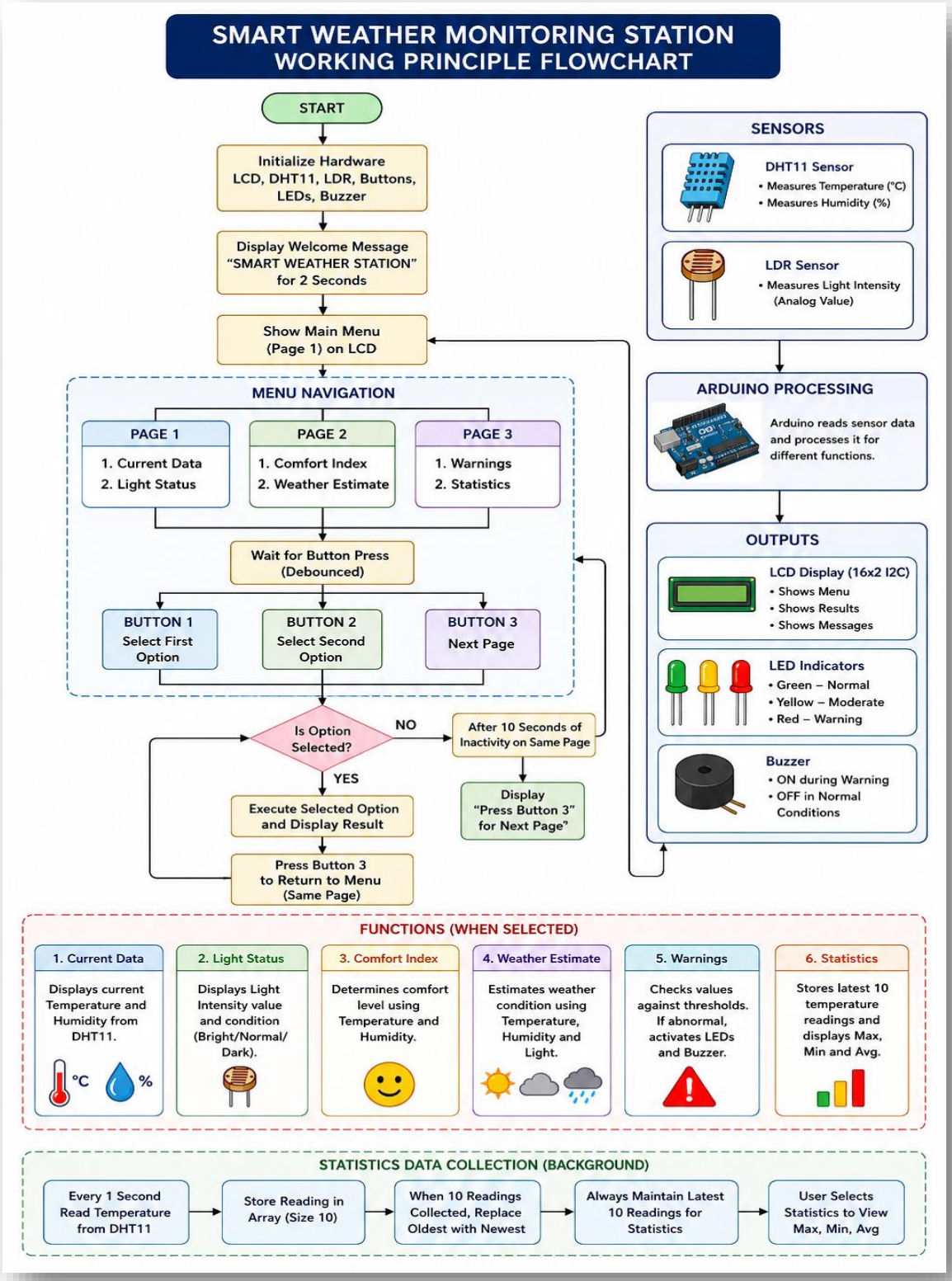
Course Name: *Embedded Systems and IoT*

Submitted By: *Aqsa Noor*

Student Status: *Student of PIEAS*

Department: *Electrical Engineering*

Submission Date: *July 2026*


SMART WEATHER MONITORING STATION
WORKING PRINCIPLE FLOWCHART
START
Initialize Hardware
LCD, DHT11, LDR, Buttons, LEDs, Buzzer
Display Welcome Message
"SMART WEATHER STATION"
for 2 Seconds
Show Main Menu
(Page 1) on LCD
MENU NAVIGATION
PAGE 1
1. Current Data
2. Light Status
PAGE 2
1. Comfort Index
2. Weather Estimate
PAGE 3
1. Warnings
2. Statistics
Wait for Button Press
(Debounced)
BUTTON 1
Select First Option
BUTTON 2
Select Second Option
BUTTON 3
Next Page
Is Option Selected?
NO
After 10 Seconds of Inactivity on Same Page
Display "Press Button 3" for Next Page"
YES
Execute Selected Option and Display Result
Press Button 3
to Return to Menu
(Same Page)
SENSORS
DHT11 Sensor
• Measures Temperature (°C)
• Measures Humidity (%)
LDR Sensor
• Measures Light Intensity (Analog Value)
ARDUINO PROCESSING
Arduino reads sensor data and processes it for different functions.
OUTPUTS
LCD Display (16x2 I2C)
• Shows Menu
• Shows Results
• Shows Messages
LED Indicators
• Green – Normal
• Yellow – Moderate
• Red – Warning
Buzzer
• ON during Warning
• OFF in Normal Conditions
FUNCTIONS (WHEN SELECTED)
1. Current Data
Displays current Temperature and Humidity from DHT11.
°C
%
2. Light Status
Displays Light Intensity value and condition (Bright/Normal/ Dark).
3. Comfort Index
Determines comfort level using Temperature and Humidity.
4. Weather Estimate
Estimates weather condition using Temperature, Humidity and Light.
5. Warnings
Checks values against thresholds. If abnormal, activates LEDs and Buzzer.
6. Statistics
Stores latest 10 temperature readings and displays Max, Min and Avg.
STATISTICS DATA COLLECTION (BACKGROUND)
Every 1 Second Read Temperature from DHT11
Store Reading in Array (Size 10)
When 10 Readings Collected, Replace Oldest with Newest
Always Maintain Latest 10 Readings for Statistics
User Selects Statistics to View Max, Min, Avg

CONTENTS

1. ABSTRACT

*This project presents a **Smart Weather Monitoring Station using Arduino** that measures temperature, humidity, and ambient light intensity using a **DHT11 sensor** and **LDR**. The system processes sensor data to display current conditions, determine comfort levels, estimate weather conditions, generate warnings, and maintain temperature statistics such as minimum, maximum, and average values. A menu-driven interface with push buttons allows users to interact with the system and access different features efficiently.*

*This project also provided valuable insight into the engineering design process. As beginners, we learned the importance of analysing system requirements, understanding the purpose of each component, selecting suitable sensors, and designing the project logic before implementation. The project introduced key embedded-system concepts such as sensor interfacing, user interaction, decision-making algorithms, and efficient programming practices, including the use of **millis()** for non-blocking operation instead of excessive **delay()** functions. Overall, the project strengthened our understanding of Arduino-based system design and provided a foundation for developing more advanced smart and IoT-enabled applications in the future.*

2. Introduction

Environmental parameters such as temperature, humidity, and light intensity play an important role in human comfort, agriculture, and industrial applications. Monitoring these conditions helps users better understand their surroundings and make informed decisions. With the advancement of embedded systems, multiple environmental parameters can now be monitored using a single microcontroller-based system.

*This project presents a **Smart Weather Monitoring Station using Arduino** that measures temperature, humidity, and ambient light conditions using a **DHT11 sensor** and **LDR**. The collected data is processed to provide useful information such as current environmental conditions, comfort levels, weather estimation, warning alerts, and temperature statistics. Beyond its functionality, the project helped develop a systematic engineering approach by emphasizing component selection, sensor interfacing, user interaction, and efficient programming techniques, providing valuable experience in embedded systems design and problem-solving.*

3. Project Objectives

- *To learn **Arduino programming and embedded systems development** through a practical and interactive project.*
- *To understand how sensors, displays, and other electronic components are selected, interfaced, and integrated into a complete system.*
- *To develop logical thinking and problem-solving skills by designing system functionality, user interaction, and data-processing algorithms before implementation.*

•*To gain hands-on experience in building a real-world monitoring system and establish a foundation for future projects in* ***IoT, automation, and smart systems****.*

4. System Overview

The proposed Weather Station is built around the Arduino microcontroller, which serves as the central processing unit of the system. It continuously collects environmental data from the DHT11 temperature and humidity sensor and the LDR (Light Dependent Resistor) sensor. The Arduino processes these sensor readings in real time and analyses the data to generate meaningful information for the user.

Based on the collected data, the system displays the current temperature, humidity, and ambient light level while also determining the comfort level, estimating weather conditions, and issuing warning alerts whenever the measured values exceed predefined thresholds. By converting raw sensor data into useful and easy-to-understand information, the Weather Station provides users with continuous environmental monitoring and enhances awareness of changing weather conditions.

5. Components Used

Arduino Uno, DHT11 Temperature and Humidity Sensor, Light Dependent Resistor (LDR), 16×2 LCD Display with I²C Module, Push Buttons, Light Emitting Diodes (LEDs), Piezo Buzzer, Resistors, Breadboard, and Jumper Wires.

6. Selection of Sensors

6.1- Suitability for a Beginner-Friendly Weather Station

The DHT11 sensor was chosen for this project because it is a reliable, beginner-friendly sensor that is capable of measuring both temperature and humidity using a single module. Since the primary objective of this weather station is to monitor basic environmental conditions and demonstrate sensor interfacing with an Arduino, the DHT11 provides all the essential functionality required. Its simple communication protocol, low power consumption, and seamless compatibility with the Arduino platform make it an excellent choice for educational and introductory embedded systems projects.

6.2- Availability and Future Expandability

Although more advanced sensors such as the DHT22 or BME280 offer higher accuracy, faster response times, and additional environmental measurements, the DHT11 was selected because it was readily available and well suited to the project's scope. As this project was developed at a beginner level, using the DHT11 allowed the focus to remain

on understanding sensor integration, data processing, and system logic rather than dealing with more complex hardware and software requirements. This approach provides a strong foundation for learning while allowing the system to be easily upgraded with more advanced sensors in future versions of the weather station.

7. Working Principle

7.1- System Initialization

When the Arduino is powered on, it initializes all the connected components, including the ***DHT11 sensor****,* ***LDR****,* ***16×2 I2C LCD****,* ***push buttons****,* ***LEDs****, and* ***buzzer****. A welcome message,* "SMART WEATHER STATION"*, is displayed on the LCD for a few seconds before the system automatically loads the main menu.*

7.2- Menu-Driven User Interface

The system uses a ***three-page menu*** *displayed on the 16×2 LCD to provide an organized and user-friendly interface.*

Page 1

- *Current Data*
- *Light Status*

Page 2

- *Comfort Index*
- *Weather Estimate*

Page 3

- *Warnings*
- *Statistics*

The menu allows the user to navigate through different features without displaying all information at once, making the interface simple and easy to use.

7.3- Push Button Operation

Three push buttons are used to interact with the system.

- ***Button 1:** Selects the **first option** displayed on the current menu page.*
- ***Button 2:** Selects the **second option** displayed on the current menu page.*
- ***Button 3:** Moves to the **next menu page**.*

*If the user remains on the same page for a predefined time, the LCD displays a reminder message such as **"Press Button 3 for Next Page"**, improving user interaction and making navigation more intuitive.*

7.4 - DHT11 Sensor Operation

*The **DHT11 sensor** continuously measures the surrounding **temperature** and **humidity**. The sensor sends digital data to the Arduino, which processes the readings and uses them in multiple system functions.*

The measured values are used to:

- *Display current temperature and humidity.*
- *Determine the comfort level.*
- *Estimate weather conditions.*
- *Generate warning messages.*
- *Calculate temperature statistics.*

7.5 - LDR Sensor Operation

*The **Light Dependent Resistor (LDR)** measures the ambient light intensity by changing its resistance according to the amount of light falling on it. The Arduino reads this value as an analog input and classifies the surroundings into:*

- ***Bright***
- ***Normal***
- ***Dark***

The light information is displayed to the user and is also used in the weather estimation process.

7.6 - Current Data

*When the user selects **Current Data**, the Arduino reads the latest temperature and humidity values from the DHT11 sensor and displays them on the LCD. This provides real-time environmental information.*

7.7 - Light Status

The Arduino reads the analog value from the LDR and determines the surrounding light condition based on predefined threshold values. Both the measured light level and its corresponding condition (Bright, Normal, or Dark) are displayed.

7.8 - Comfort Index

*The Arduino processes both **temperature** and **humidity** readings to determine the environmental comfort level.*

The environment is classified as:

- *Comfortable*
- *Moderate*
- *Hot*
- *Cold*

This feature converts raw sensor data into meaningful information that is easier for the user to understand.

7.9 - Weather Estimation

*The weather estimation feature combines the **temperature**, **humidity**, and **light intensity** to provide a simple prediction of environmental conditions.*

Depending on the sensor readings, the system estimates conditions such as:

- *Sunny*
- *Cloudy*
- *Rain Possible*
- *Normal*

Although it is not a professional weather forecast, it demonstrates how multiple sensors can be used together to make logical decisions.

7.10 - Warning System

The Arduino continuously compares sensor readings with predefined safety limits.

If abnormal conditions are detected, warning messages such as:

- *High Temperature*
- *High Humidity*
- *Low Light*

are displayed on the LCD.

At the same time:

- *The **Red LED** turns ON to indicate a warning.*
- *The **Buzzer** produces an audible alert to immediately notify the user.*

If all environmental conditions remain within safe limits:

- *The **Green LED** indicates normal operation.*
- *The buzzer remains OFF.*

7.11 - LED Indication

Three LEDs provide a quick visual indication of the system status.

- ***Green LED:** Safe and normal environmental conditions.*
- ***Yellow LED:** Moderate conditions that may require attention.*
- ***Red LED:** Warning condition when predefined thresholds are exceeded.*

These indicators allow the user to understand the system status instantly without reading the LCD.

7.12 - Buzzer Operation

The buzzer serves as an audible warning device. It remains OFF during normal operation and is activated whenever the Arduino detects unsafe environmental

conditions, such as excessive temperature or humidity. This ensures that important alerts are immediately noticeable.

7.13 -Statistics Calculation

*The Arduino continuously stores the latest **10 temperature readings** collected from the DHT11 sensor.*

Using these readings, it calculates:

- *Maximum Temperature*
- *Minimum Temperature*
- *Average Temperature*

*Whenever the user selects the **Statistics** option, the Arduino processes the stored readings and displays these values on the LCD. As new sensor readings are collected, the oldest values are replaced, ensuring that the statistics always represent the most recent environmental conditions.*

---

8. Communication Protocol Used

8.1 - What is a Communication Protocol?

*A **communication protocol** is a set of predefined rules that allows two or more electronic devices to exchange data accurately and reliably. It defines how information is transmitted, received, and interpreted so that both devices understand each other correctly. Without a communication protocol, devices would not know how to send or decode data, resulting in communication errors. In this project, communication protocols enable the Arduino to receive data from sensors and send information to the LCD for display.*

8.2 - DHT11 Communication Protocol

Protocol Used: Single-Wire Digital Communication Protocol

*The DHT11 sensor communicates with the Arduino using a **Single-Wire Digital Communication Protocol**, which requires only one data pin in addition to power and ground connections. The Arduino first sends a start signal to the sensor, after which the DHT11 responds by transmitting digital temperature and humidity data as a sequence of*

binary bits (0s and 1s). The DHT library decodes this digital data into readable temperature and humidity values that are processed and displayed by the Arduino.

Advantages

- *Uses only one data wire, reducing wiring complexity.*
- *Provides digital output, eliminating the need for analog conversion.*
- *Simple to interface, making it ideal for beginner-level projects.*

8.3 - LCD Communication Protocol (I2C)

Protocol Used: I2C (Inter-Integrated Circuit)

*The 16×2 LCD is connected to the Arduino through an **I2C module**, which communicates using only two signal lines: **SDA (Serial Data)** and **SCL (Serial Clock)**. The Arduino sends commands and display data through these two lines, while the clock signal synchronizes the communication between the Arduino and the LCD. Compared to a standard LCD connection, the I2C module significantly reduces the number of required Arduino pins and simplifies the overall circuit.*

Advantages

- *Uses only two communication lines (SDA and SCL).*
- *Reduces wiring and saves Arduino input/output pins.*
- *Allows multiple I2C devices to share the same communication bus using unique addresses.*

8.4 - Communication Protocols Used in This Project

Component	***Communication Protocol***	***Purpose***
DHT11 Sensor	*Single-Wire Digital Communication*	*Sends digital temperature and humidity data to the Arduino through a single data line.*
16×2 LCD with I2C Module	*I2C (Inter-Integrated Circuit)*	*Receives processed information from the Arduino and displays it using only SDA and SCL communication lines.*

8.5 - Importance of Communication Protocols in This Project

*Communication protocols play a vital role in ensuring reliable and efficient data exchange between the Arduino and connected devices. The **Single-Wire protocol** enables the DHT11 sensor to transmit accurate environmental data using minimal wiring,* while *the **I2C protocol** allows the LCD to display information using only two communication lines. Together, these protocols simplify circuit design, reduce hardware complexity, save Arduino pins, and improve the overall efficiency and reliability of the Smart Weather Monitoring Station.*

9. Arduino Processing and Logic

*The **Arduino Uno** acts as the central processing unit (CPU) of the Smart Weather Monitoring Station. It continuously receives input from the **DHT11 sensor** and **LDR**, processes the collected data according to the programmed logic, and controls the system outputs. Based on the sensor readings, the Arduino displays real-time temperature, humidity, and light intensity on the LCD, calculates the **Comfort Index**, estimates weather conditions, generates warning messages, and updates temperature statistics such as **minimum**, **maximum**, and **average** values.*

*The Arduino also manages all **user interactions** through the three push buttons. It controls the menu-driven interface, detects button presses using software debouncing with the `millis()` function, and executes the selected option without blocking the program. At the same time, it controls the **LEDs** and **buzzer** to provide visual and audible alerts whenever predefined environmental thresholds are exceeded. By continuously repeating this process, the Arduino ensures that the weather station operates efficiently, responds quickly to user inputs, and provides accurate real-time environmental monitoring.*

10. System Design, Implementation and Performance

10.1 - System Design

The Smart Weather Monitoring Station is designed as a menu-driven embedded system using an Arduino Uno as the main controller. The system integrates a DHT11 sensor for measuring temperature and humidity, an LDR for detecting ambient light intensity, a 16×2 I2C LCD for displaying information, three push buttons for user interaction, and LEDs with a buzzer for visual and audible alerts. The Arduino continuously processes sensor data and presents meaningful information through an organized menu interface.

10.2 - System Features

The weather station provides several intelligent features that enhance environmental monitoring, including:

- *Real-time temperature and humidity display.*
- *Ambient light level classification (Bright, Normal, Dark).*
- *Comfort Index calculation based on temperature and humidity.*
- *Weather estimation using combined sensor data.*
- *Automatic warning generation for abnormal environmental conditions.*
- *Temperature statistics including minimum, maximum, and average values.*
- *Interactive menu-driven navigation using push buttons.*

10.3 - Implementation and Testing

Each hardware component was initially tested independently to verify its operation. The DHT11 sensor, LDR, LCD, push buttons, LEDs, and buzzer were first validated using the Arduino Serial Monitor. After successful individual testing, all components were integrated into a single program. The complete system was then tested to verify menu navigation, sensor readings, warning generation, statistical calculations, and overall user interaction.

10.4 - System Performance and Observations

The Smart Weather Monitoring Station operated successfully by continuously monitoring environmental conditions and processing sensor data in real time. The menu-driven interface responded correctly to user inputs, the LCD displayed accurate information, and the LEDs and buzzer provided timely visual and audible warnings whenever predefined thresholds were exceeded. The statistics feature correctly maintained the latest temperature readings and calculated the minimum, maximum, and average temperatures, demonstrating reliable and efficient system performance.

11. Possible Integration and Future Scope

11.1 - IoT-Based Remote Monitoring (ESP32)

*By integrating an **ESP32**, the system can transmit real-time sensor data to a mobile application or web dashboard, enabling remote monitoring through Wi-Fi.*

11.2 - Smart Home Automation

The weather station can automatically control appliances such as fans, lights, and exhaust systems based on temperature, humidity, and light conditions, improving comfort and energy efficiency.

11.3 - Industrial and Agricultural Monitoring

*The system can be deployed in **factories, laboratories, greenhouses, and agricultural fields** to continuously monitor environmental conditions, ensuring safety, productivity, and optimal operating conditions.*

11.4 - Artificial Intelligence and Machine Learning

*By incorporating **AI and Machine Learning**, the system can analyse historical data, predict environmental trends, detect abnormal conditions, and make more intelligent decisions than simple threshold-based systems.*

12. Challenges Faced

*As a beginner in Arduino programming, one of the most significant challenges was developing the **logical flow of the program**. Designing a menu-driven system required careful planning of user interaction, sensor processing, decision-making, and navigation between different menu options. Building this logic step by step helped in understanding how embedded systems process information and respond to user inputs.*

*Another challenge was **hardware interfacing and troubleshooting**. During development, sensor connections, LCD wiring, and push-button circuits had to be checked repeatedly to ensure that errors were caused by the program rather than incorrect hardware connections. This debugging process improved practical skills in circuit verification, systematic problem-solving, and hardware-software integration.*

13. *Conclusion*

*The **Smart Weather Monitoring Station** successfully demonstrates the practical implementation of an embedded system by integrating sensors, user interaction, and logical decision-making into a single project. Using the **Arduino Uno**, **DHT11 sensor**, **LDR**, **16×2 I2C LCD**, **push buttons**, **LEDs**, and **buzzer**, the system continuously monitors environmental conditions and provides meaningful information through a*

menu-driven interface, including current data, light status, comfort index, weather estimation, warnings, and temperature statistics.

Beyond achieving its functional objectives, this project served as an excellent learning experience in Arduino programming, sensor interfacing, communication protocols, hardware integration, and embedded system design. It strengthened the understanding of how to approach a real-world engineering project by planning the system, developing logical program flow, testing individual modules, and integrating them into a complete solution. Furthermore, the project provides a strong foundation for future enhancements such as IoT connectivity, smart home automation, industrial monitoring, and Artificial Intelligence-based environmental analysis, making it a scalable platform for advanced embedded system applications.

14. References

Arduino Documentation, DHT11 Datasheet, LDR Datasheet and LiquidCrystal_I2C Documentation.

PROJECT BREADBOARD PROTOTYPE